

```

1  -- E-Commerce End to End Project
2  -- First Leg of the Project of Data Manipulation and Cleaning will be in PostgreSQL.
3
4  -- Creating table customers
5  DROP TABLE IF EXISTS customers;
6  Create Table customers
7  (
8      customer_id          INT PRIMARY KEY,
9      full_name             VARCHAR(100),
10     age                   INT,
11     city                  VARCHAR(50),
12     gender                VARCHAR(10),
13     state                 VARCHAR(50),
14     yearly_income         DECIMAL(10,2),
15     education             VARCHAR(50),
16     occupation            VARCHAR(50)
17 );
18
19 -- Copy the data from csv file to customers table
20 COPY customers(customer_id, full_name, age, city, gender, state, yearly_income, education, occupation)
21 FROM 'C:\\Program Files\\PostgreSQL\\17\\data\\customers.csv'
22 DELIMITER ','
23 CSV HEADER;
24
25 -- Query customers table
26 SELECT *
27 FROM customers
28 LIMIT 5;
29
30 -- Creating transactions table
31 DROP TABLE IF EXISTS transactions;
32 Create Table transactions
33 (
34     transaction_id        INT PRIMARY KEY,
35     session_id            INT,
36     customer_id           INT,
37     purchase_amount       DECIMAL(10, 2),
38     product_category      VARCHAR(50),
39     payment_method        VARCHAR(50),
40     transaction_date       TIMESTAMP
41 );
42
43 -- Adding data to transactions table from csv file
44 COPY transactions(transaction_id, session_id, customer_id, purchase_amount, product_category, payment_method, transaction_date)
45 FROM 'C:\\Program Files\\PostgreSQL\\17\\data\\transactions.csv'
46 DELIMITER ','
47 CSV HEADER;
48
49 -- Query first 5 rows of transactions table
50 SELECT *
51 FROM transactions
52 LIMIT 5;
53
54 -- Creating churn table
55 DROP TABLE IF EXISTS churn;
56 Create Table churn
57 (
58     customer_id           INT,
59     total_orders          INT,
60     days_since_last_purchase INT,
61     last_purchase_date     TIMESTAMP
62 );
63
64 -- Adding data to transactions table from csv file
65 COPY churn(customer_id, total_orders, days_since_last_purchase, last_purchase_date)
66 FROM 'C:\\Program Files\\PostgreSQL\\17\\data\\churn.csv'
67 DELIMITER ','
68 CSV HEADER;
69
70 -- Query first 5 rows of churn table
71 SELECT *
72 FROM churn
73 LIMIT 5;
74
75 -- Creating session_events table
76 DROP TABLE IF EXISTS session_events;
77 Create Table session_events
78 (
79     session_id            INT,

```

```

80     customer_id      INT,
81     event_type       VARCHAR(20),
82     timestamp        TIMESTAMPT
83 );
84
85 -- Adding data to session_events table from csv file
86 COPY session_events(session_id, customer_id, event_type, timestamp)
87 FROM 'C:\\Program Files\\PostgreSQL\\17\\data\\session_events.csv'
88 DELIMITER ','
89 CSV HEADER;
90
91 -- Query first 5 rows of session_events table
92 SELECT *
93 FROM session_events
94 LIMIT 5;
95
96
97 -- Exploring customers table
98
99 -- Total customers
100 SELECT COUNT(*)
101 FROM customers;
102
103 -- Total unique customers
104 SELECT COUNT(DISTINCT(full_name, age, city, gender, state, yearly_income, education, occupation))
105 FROM customers;
106
107 -- Checking for Duplicate rows
108 SELECT DISTINCT(full_name, age, city, gender, state, yearly_income, education, occupation),
109                COUNT(1) AS frequency
110 FROM customers
111 GROUP BY DISTINCT(full_name, age, city, gender, state, yearly_income, education, occupation)
112 ORDER BY frequency DESC;
113
114 -- Removing Duplicate records from customers table
115 DELETE FROM customers
116 WHERE customer_id IN
117 (
118     SELECT customer_id
119     FROM (
120         SELECT *,
121              ROW_NUMBER() OVER(PARTITION BY full_name, age, city, gender, state, yearly_income, education, occupation) AS rn
122         FROM customers
123     ) AS t
124     WHERE rn > 1
125 );
126
127 -- Checking for missing values
128 SELECT COUNT(*) - COUNT(full_name) AS full_name_missing,
129        COUNT(*) - COUNT(age) AS age_missing,
130        COUNT(*) - COUNT(city) AS city_missing,
131        COUNT(*) - COUNT(gender) AS gender_missing,
132        COUNT(*) - COUNT(state) AS state_missing,
133        COUNT(*) - COUNT(yearly_income) AS yearly_income_missing,
134        COUNT(*) - COUNT(education) AS education_missing,
135        COUNT(*) - COUNT(occupation) AS occupation_missing
136 FROM customers;
137
138 -- Checking out age column
139 SELECT MIN(age) AS youngest_customer,
140        MAX(age) AS oldest_customer,
141        ROUND(AVG(age),2) AS avg_age,
142        PERCENTILE_CONT(0.5) WITHIN GROUP(ORDER BY age) AS median_age
143 FROM customers;
144
145 -- Checking out outliers in age column
146 WITH quartiles AS (
147     SELECT
148         PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY age) AS q1,
149         PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY age) AS q3
150     FROM customers
151 )
152 SELECT *
153 FROM customers, quartiles
154 WHERE age > (q3 + 1.5 * (q3 - q1)) -- Upper fence
155        OR age < (q1 - 1.5 * (q3 - q1)) -- Lower fence
156 ORDER BY age DESC;
157
158 -- Checking the age column, we have found some rows with customers age as 120.
159 -- We will remove all customers above age of 100 because these are extreme outliers.
160 DELETE FROM customers

```

```

161 WHERE age > 100;
162
163 -- Checking out city column
164 SELECT city,
165         COUNT(1) AS number_of_customers
166 FROM customers
167 GROUP BY city
168 ORDER BY number_of_customers DESC;
169
170 -- Checking out gender column
171 SELECT gender,
172         COUNT(1) AS number_of_customers
173 FROM customers
174 GROUP BY gender
175 ORDER BY number_of_customers DESC;
176
177 -- Checking out state column
178 SELECT state,
179         COUNT(1) AS number_of_customers
180 FROM customers
181 GROUP BY state
182 ORDER BY number_of_customers DESC;
183
184 -- Checking out yearly_income column
185 SELECT MIN(yearly_income) AS youngest_customer,
186        MAX(yearly_income) AS oldest_customer,
187        ROUND(AVG(yearly_income),2) AS avg_yearly_income,
188        PERCENTILE_CONT(0.5) WITHIN GROUP(ORDER BY yearly_income) AS median_yearly_income
189 FROM customers;
190
191 -- Checking out education column
192 SELECT education,
193         COUNT(1) AS number_of_customers
194 FROM customers
195 GROUP BY education
196 ORDER BY number_of_customers DESC;
197
198 -- Checking out occupation column
199 SELECT occupation,
200         COUNT(1) AS number_of_customers
201 FROM customers
202 GROUP BY occupation
203 ORDER BY number_of_customers DESC;
204
205 -- checking combination of city and state
206 SELECT city,
207        state,
208        COUNT(1) AS num_of_customers
209 FROM customers
210 GROUP BY 1, 2
211 ORDER BY state;
212
213 -- Updating city column based on values from city, state combinations given here
214 -- Delhi state can only have Delhi has city
215 -- State of Gujarat only has data for Ahmedabad city
216 -- State of Karnataka only has data for Bangalore city
217 -- State of Rajasthan only has data for Jaipur city
218 -- State of Telangana only has data for Hyderabad city
219 -- State of Uttar Pradesh only has data for Lucknow city
220 -- State of Tamil Nadu only has data for Chennai city
221 -- State of West Bengal only has data for Kolkata city
222 -- State of Maharashtra has data for two cities Pune and Mumbai, so based on percentage of occurrence we are going to fill cities.
223
224 -- Replacing null with Delhi
225 UPDATE customers
226 SET city = 'Delhi'
227 WHERE city IS NULL AND state = 'Delhi';
228
229 -- Replacing null with Ahmedabad
230 UPDATE customers
231 SET city = 'Ahmedabad'
232 WHERE city IS NULL AND state = 'Gujarat';
233
234 -- Replacing null with Bangalore
235 UPDATE customers
236 SET city = 'Bangalore'
237 WHERE city IS NULL AND state = 'Karnataka';
238
239 -- Replacing null with Jaipur
240 UPDATE customers
241 SET city = 'Jaipur';

```

```

242 WHERE city IS NULL AND state = 'Rajasthan';
243
244 -- Replacing null with Hyderabad
245 UPDATE customers
246 SET city = 'Hyderabad'
247 WHERE city IS NULL AND state = 'Telangana';
248
249 -- Replacing null with Chennai
250 UPDATE customers
251 SET city = 'Chennai'
252 WHERE city IS NULL AND state = 'Tamil Nadu';
253
254 -- Replacing null with Lucknow
255 UPDATE customers
256 SET city = 'Lucknow'
257 WHERE city IS NULL AND state = 'Uttar Pradesh';
258
259 -- Replacing null with West Bengal
260 UPDATE customers
261 SET city = 'Kolkata'
262 WHERE city IS NULL AND state = 'West Bengal';
263
264 -- Filling Maharashtra values
265 SELECT state,
266        city,
267        (COUNT(*) :: NUMERIC)/ (SELECT COUNT(*) FROM customers WHERE state='Maharashtra') * 100
268 FROM customers
269 WHERE state = 'Maharashtra'
270 GROUP BY 1,2;
271
272 -- Seeing that Mumbai is having 75% proportion in Maharashtra and rest in Pune,
273 -- we will fill those 77 missing values based on this proportion.
274 -- so, we will fill first 57 values with Mumbai and 20 with Pune city.
275
276 WITH CTE AS (
277     SELECT
278         customer_id,
279         city, state,
280         ROW_NUMBER() OVER (ORDER BY customer_id) AS rn
281     FROM customers
282     WHERE city IS NULL AND state = 'Maharashtra'
283 )
284
285 UPDATE customers
286 SET city = CASE
287             WHEN rn <= 57 THEN 'Mumbai'
288             ELSE 'Pune'
289         END
290 FROM CTE
291 WHERE customers.city IS NULL
292 AND customers.state = 'Maharashtra'
293 AND customers.customer_id = CTE.customer_id;
294
295 -- Checking out state column to fill values
296 SELECT city,
297        state,
298        COUNT(1) AS num_of_customers
299 FROM customers
300 GROUP BY 1, 2
301 ORDER BY state;
302
303 -- Based on state data, we would fill state null values from city names.
304 -- Delhi state can only have Delhi has city,
305 -- Ahmedabad is in Gujarat,
306 -- Bangalore is in Karnataka,
307 -- Jaipur is in Rajasthan,
308 -- Hyderabad is in Telangana,
309 -- Lucknow is in Uttar Pradesh,
310 -- Chennai is in Tamil Nadu,
311 -- Kolkata is in West Bengal
312 -- Pune and Mumbai are in Maharashtra
313
314 -- Replacing null in state of Delhi
315 UPDATE customers
316 SET state = 'Delhi'
317 WHERE state IS NULL
318 AND city = 'Delhi';
319
320 -- Replacing null in state of Gujarat
321 UPDATE customers
322 SET state = 'Gujarat'

```

```

323 WHERE state IS NULL
324     AND city = 'Ahmedabad';
325
326 -- Replacing null in state of Karnataka
327 UPDATE customers
328 SET state = 'Karnataka'
329 WHERE state IS NULL
330     AND city = 'Bangalore';
331
332 -- Replacing null in state of Rajasthan
333 UPDATE customers
334 SET state = 'Rajasthan'
335 WHERE state IS NULL
336     AND city = 'Jaipur';
337
338 -- Replacing null in state of Telangana
339 UPDATE customers
340 SET state = 'Telangana'
341 WHERE state IS NULL
342     AND city = 'Hyderabad';
343
344 -- Replacing null in state of Uttar Pradesh
345 UPDATE customers
346 SET state = 'Uttar Pradesh'
347 WHERE state IS NULL
348     AND city = 'Lucknow';
349
350 -- Replacing null in state of Tamil Nadu
351 UPDATE customers
352 SET state = 'Tamil Nadu'
353 WHERE state IS NULL
354     AND city = 'Chennai';
355
356 -- Replacing null in state of West Bengal
357 UPDATE customers
358 SET state = 'West Bengal'
359 WHERE state IS NULL
360     AND city = 'Kolkata';
361
362 -- Replacing null in state of Maharashtra
363 UPDATE customers
364 SET state = 'Maharashtra'
365 WHERE state IS NULL
366     AND (city = 'Pune' OR city = 'Mumbai');
367
368 -- Still found that city and state both have 3 same missing values,
369 -- we will check the combination of education, occupation, and city with count of customers,
370 -- and fill those values in customer table.
371 WITH CTE AS(
372     SELECT education,
373            occupation,
374            city
375     FROM (
376         SELECT education,
377                occupation,
378                city,
379                ROW_NUMBER() OVER(PARTITION BY education, occupation ORDER BY COUNT(1) DESC) AS rank
380         FROM customers
381         GROUP BY 1,2,3 ) AS t
382     WHERE rank = 1
383 )
384
385 UPDATE customers
386 SET city = CTE.city
387 FROM CTE
388 WHERE customers.education = CTE.education
389     AND customers.occupation = CTE.occupation
390     AND customers.city IS NULL;
391
392 -- Finding which state has missing values to check for distinct city names
393 SELECT DISTINCT city
394 FROM customers
395 WHERE state IS NULL;
396
397 -- Filling state missing values for city Bangalore
398 UPDATE customers
399 SET state = 'Karnataka'
400 WHERE state IS NULL
401     AND city = 'Bangalore';
402
403 -- Filling state missing values for city Ahmedabad

```

```

404 UPDATE customers
405 SET state = 'Gujarat'
406 WHERE state IS NULL
407         AND city = 'Ahmedabad';
408
409 -- Filling null values in age column
410 -- Checking mean age per gender
411 SELECT gender,
412        COUNT(*),
413        AVG(age)
414 FROM customers
415 GROUP BY gender;
416
417 -- Checking mean age per city
418 SELECT city, AVG(age)
419 FROM customers
420 GROUP BY city;
421
422 -- Age is similar across gender and city, so we will replace age null values with mean of age column.
423 UPDATE customers
424 SET age = (SELECT ROUND(AVG(age),0) FROM customers)
425 WHERE age IS NULL;
426
427 -- Replacing the null values in yearly_income column based on mean values per occupation, city group.
428 -- This is done because the place of work and occupation matters for yearly income.
429 WITH CTE AS (
430 SELECT
431         occupation,
432         city,
433         ROUND(AVG(yearly_income)) AS avg_yearly_income
434 FROM customers
435 WHERE occupation IS NOT NULL
436         AND city IS NOT NULL
437 GROUP BY occupation, city
438 )
439
440 UPDATE customers
441 SET yearly_income = CTE.avg_yearly_income
442 FROM CTE
443 WHERE customers.occupation = CTE.occupation
444        AND customers.city = CTE.city
445        AND customers.yearly_income IS NULL;
446
447 -- Checking for missing values again in yearly income column
448 SELECT occupation, city, yearly_income
449 FROM customers
450 WHERE yearly_income IS NULL;
451
452 -- Upon checking there are still some missing values for yearly_income in customers table
453 -- We will fill on the basis of mean yearly income for that occupation (if available),
454 -- then fill based on city's mean yearly income.
455 -- Filling null based on occupation first
456 WITH CTE AS (
457 SELECT
458         occupation,
459         ROUND(AVG(yearly_income)) AS avg_yearly_income
460 FROM customers
461 WHERE occupation IS NOT NULL
462 GROUP BY occupation
463 )
464
465 UPDATE customers
466 SET yearly_income = CTE.avg_yearly_income
467 FROM CTE
468 WHERE customers.occupation = CTE.occupation
469        AND customers.yearly_income IS NULL;
470
471 -- Fillin null in yearly_income based on city
472 WITH CTE AS (
473 SELECT
474         city,
475         ROUND(AVG(yearly_income)) AS avg_yearly_income
476 FROM customers
477 WHERE city IS NOT NULL
478 GROUP BY city
479 )
480
481 UPDATE customers
482 SET yearly_income = CTE.avg_yearly_income
483 FROM CTE
484 WHERE customers.city = CTE.city

```

```

485 AND customers.yearly_income IS NULL;
486
487 -- For filling missing values in education column, checked for different column wise data.
488 -- Decided to go through with grouped by values for each city, occupation combination with most likely educational qualifications.
489 -- For example, for city of Ahmedabad, most Accountants are having a B.Com education, so we will fill those null values in this
490 -- city, occupation combination with B.Com, and so no.
491 WITH CTE AS (
492     SELECT city,
493            occupation,
494            education
495     FROM (
496         SELECT city,
497                occupation,
498                education,
499                ROW_NUMBER() OVER(PARTITION BY city, occupation ORDER BY COUNT(1) DESC) AS rank
500         FROM customers
501         WHERE occupation IS NOT NULL
502               AND education IS NOT NULL
503         GROUP BY 1, 2, 3) AS t
504     WHERE rank = 1
505 )
506
507 UPDATE customers
508 SET education = CTE.education
509 FROM CTE
510 WHERE customers.city = CTE.city
511        AND customers.occupation = CTE.occupation
512        AND customers.education IS NULL;
513
514 -- Checking remaining missing values in education column
515 SELECT *
516 FROM customers
517 WHERE education IS NULL;
518
519 -- First we will fill null values in occupation then update the education column
520 -- Filling null values in occupation based on city wise most occurring occupation for customers
521 WITH CTE AS (
522     SELECT city,
523            occupation
524     FROM (
525         SELECT city,
526                occupation,
527                ROW_NUMBER() OVER(PARTITION BY city ORDER BY COUNT(1) DESC) AS rank
528         FROM customers
529         WHERE occupation IS NOT NULL
530         GROUP BY 1, 2) AS t
531     WHERE rank = 1
532 )
533
534 UPDATE customers
535 SET occupation = CTE.occupation
536 FROM CTE
537 WHERE customers.city = CTE.city
538        AND customers.occupation IS NULL;
539
540
541 -- Running the previous Update query again to fill null values in education where occupation was missing
542 WITH CTE AS (
543     SELECT city,
544            occupation,
545            education
546     FROM (
547         SELECT city,
548                occupation,
549                education,
550                ROW_NUMBER() OVER(PARTITION BY city, occupation ORDER BY COUNT(1) DESC) AS rank
551         FROM customers
552         WHERE occupation IS NOT NULL
553               AND education IS NOT NULL
554         GROUP BY 1, 2, 3) AS t
555     WHERE rank = 1
556 )
557
558 UPDATE customers
559 SET education = CTE.education
560 FROM CTE
561 WHERE customers.city = CTE.city
562        AND customers.occupation = CTE.occupation
563        AND customers.education IS NULL;
564
565 -- Checking for missing values

```

```

566 SELECT COUNT(*) - COUNT(full_name) AS full_name_missing,
567        COUNT(*) - COUNT(age) AS age_missing,
568        COUNT(*) - COUNT(city) AS city_missing,
569        COUNT(*) - COUNT(gender) AS gender_missing,
570        COUNT(*) - COUNT(state) AS state_missing,
571        COUNT(*) - COUNT(yearly_income) AS yearly_income_missing,
572        COUNT(*) - COUNT(education) AS education_missing,
573        COUNT(*) - COUNT(occupation) AS occupation_missing
574 FROM customers;
575
576 -- Exploring transactions table
577
578 -- Remove all transactions before 2024-03-19
579 DELETE FROM transactions
580 WHERE transaction_date < '2024-03-19';
581
582 -- Querying first 5 rows
583 SELECT *
584 FROM transactions
585 LIMIT 5;
586
587 -- Check for number of records
588 SELECT COUNT(*)
589 FROM transactions;
590
591 -- Check for duplicate rows
592 SELECT COUNT(DISTINCT(session_id, transaction_date))
593 FROM transactions;
594
595 -- Checking for missing values
596 SELECT
597        COUNT(*) - COUNT(customer_id) AS customer_id_missing,
598        COUNT(*) - COUNT(purchase_amount) AS purchase_amount_missing,
599        COUNT(*) - COUNT(product_category) AS product_category_missing,
600        COUNT(*) - COUNT(payment_method) AS payment_method_missing,
601        COUNT(*) - COUNT(transaction_date) AS transaction_date_missing
602 FROM transactions;
603
604 -- There are no missing values in transactions table
605
606 -- Let us start exploring the dataset
607 -- Exploring customer_id column
608 SELECT COUNT(DISTINCT(customer_id)) AS customers_that_placed_order
609 FROM transactions;
610
611 -- Exploring purchase_amount column
612 SELECT MIN(purchase_amount) AS min_purchase_amount,
613        AVG(purchase_amount) AS avg_purchase_amount,
614        PERCENTILE_CONT(0.5) WITHIN GROUP(ORDER BY purchase_amount) AS median_purchase_amount,
615        MAX(purchase_amount) AS max_purchase_amount
616 FROM transactions;
617
618 -- Detecting outliers in purchase_amount column so will check for that
619 SELECT q3 + (1.5 * (q3-q1)) as upper_bound,
620        GREATEST(q1 - (1.5 * (q3-q1)), 0) as lower_bound
621 FROM (
622        SELECT PERCENTILE_CONT(0.75) WITHIN GROUP(ORDER BY purchase_amount) as q3,
623               PERCENTILE_CONT(0.25) WITHIN GROUP(ORDER BY purchase_amount) as q1
624        FROM transactions) AS t;
625
626 -- Seeing the upper and lower bound and median of purchase_amount, we can say that there are no extreme outliers in the column.
627
628 -- Exploring purchase_category column
629 SELECT product_category,
630        COUNT(*) AS transaction_count
631 FROM transactions
632 GROUP By product_category
633 ORDER BY transaction_count DESC;
634
635 -- There are 9 unique category of products and customers purchased Luxury Fashion category most of the times.
636 -- Also, they purchased from books category of products the least.
637
638 -- Exploring payment_method column
639 SELECT payment_method,
640        COUNT(*) AS transaction_count
641 FROM transactions
642 GROUP By payment_method
643 ORDER BY transaction_count DESC;
644
645 -- Exploring transaction_date column
646 -- Monthly transactions

```



```

647 SELECT EXTRACT(MONTH FROM transaction_date) AS month,
648         COUNT(*) AS num_txns
649 FROM transactions
650 GROUP BY month
651 ORDER BY month;
652
653 -- Yearly transactions
654 SELECT EXTRACT(year FROM transaction_date) AS year,
655         COUNT(*) AS num_txns
656 FROM transactions
657 GROUP BY year
658 ORDER BY year;
659
660 -- transactions day wise
661 SELECT TO_CHAR(transaction_date, 'Day') AS day_of_week,
662         COUNT(*) AS num_txns
663 FROM transactions
664 GROUP BY day_of_week;
665
666 -- Exploring churn table
667 -- Query first 5 rows
668 SELECT *
669 FROM churn
670 LIMIT 5;
671
672 -- Counting records
673 SELECT COUNT(*)
674 FROM churn;
675
676 -- total orders column
677 SELECT SUM(total_orders) AS sum_of_total_orders
678 FROM churn;
679
680 -- total orders stats
681 SELECT MIN(total_orders) AS minimum_total_orders,
682        ROUND(AVG(total_orders),2) AS avg_total_orders,
683        PERCENTILE_DISC(0.5) WITHIN GROUP(ORDER BY total_orders) AS median_total_orders,
684        MAX(total_orders) AS maximum_total_orders
685 FROM churn;
686
687 -- days_since_last_purchase column
688 SELECT MIN(days_since_last_purchase) AS minimum_days_since_last_purchase,
689        ROUND(AVG(days_since_last_purchase),2) AS avg_days_since_last_purchase,
690        PERCENTILE_DISC(0.5) WITHIN GROUP(ORDER BY days_since_last_purchase) AS median_days_since_last_purchase,
691        MAX(days_since_last_purchase) AS maximum_days_since_last_purchase
692 FROM churn;
693
694 -- Checking for duplicates
695 SELECT COUNT(DISTINCT(customer_id, total_orders, days_since_last_purchase, last_purchase_date))
696 FROM churn;
697
698 -- Checking for missing values
699 SELECT
700         COUNT(*) - COUNT(customer_id) AS customer_id_missing,
701         COUNT(*) - COUNT(total_orders) AS total_orders_missing,
702         COUNT(*) - COUNT(days_since_last_purchase) AS days_since_last_purchase_missing,
703         COUNT(*) - COUNT(last_purchase_date) AS last_purchase_date_missing
704 FROM churn;
705
706
707 -- SESSION_EVENT TABLE
708
709 -- Remove all session events before 2024-03-19
710 DELETE FROM session_events
711 WHERE timestamp < '2024-03-19';
712
713 -- Query first 5 rows
714 SELECT *
715 FROM session_events
716 LIMIT 5;
717
718 -- Found Duplicate rows
719 SELECT COUNT(*) - COUNT(DISTINCT(customer_id, event_type, timestamp)) AS duplicates
720 FROM session_events
721 WHERE timestamp IS NOT NULL;
722
723 -- Found 3 duplicate rows,
724 -- Solving this issue by removing exact duplicates
725
726 -- Checking rows that are duplicated without null values in timestamp
727 WITH dup AS (

```

```

728 SELECT
729     session_id,
730     customer_id,
731     event_type,
732     timestamp,
733     COUNT(*) OVER (PARTITION BY session_id) AS event_count,
734     ROW_NUMBER() OVER (
735         PARTITION BY customer_id, event_type, timestamp
736         ORDER BY ctid
737     ) AS rn
738 FROM session_events
739 WHERE timestamp IS NOT NULL
740 )
741 SELECT
742     session_id,
743     customer_id,
744     event_type,
745     timestamp,
746     event_count
747 FROM dup
748 WHERE rn > 1
749 ORDER BY customer_id, session_id;
750
751 -- Deleting the duplicated rows
752 WITH to_delete AS (
753     SELECT
754         ctid,
755         ROW_NUMBER() OVER (
756             PARTITION BY customer_id, event_type, timestamp
757             ORDER BY ctid
758         ) AS rn
759     FROM session_events
760     WHERE timestamp IS NOT NULL
761 )
762 DELETE FROM session_events
763 WHERE ctid IN (
764     SELECT ctid
765     FROM to_delete
766     WHERE rn > 1
767 );
768
769 -- Checking for Sessions with more than one unique events.
770
771 -- Finding those sessions and deleting the duplicate event.
772 WITH duplicate_events AS (
773     SELECT ctid,
774         ROW_NUMBER() OVER(PARTITION BY session_id, customer_id, event_type ORDER BY timestamp DESC) AS rn
775 FROM session_events
776 WHERE session_id IN (
777     SELECT session_id
778     FROM session_events
779     GROUP BY session_id, customer_id, event_type
780     HAVING COUNT(*) > 1)
781 )
782
783 -- Deleting those rows
784 DELETE FROM session_events
785 WHERE ctid IN (
786     SELECT ctid
787     FROM duplicate_events
788     WHERE rn > 1);
789
790 -- Deleted in total 2082 such rows.
791
792 -- Checking for Sessions with more than one customer_id
793
794 -- Upon Checking, found no such rows.
795 SELECT session_id,
796     COUNT(DISTINCT customer_id) AS distinct_customer_count
797 FROM session_events
798 GROUP BY session_id
799 HAVING COUNT(DISTINCT customer_id) > 1
800 ORDER BY distinct_customer_count DESC;
801
802 -- All duplicates have been dealt with.
803
804 -- Checking for Sessions with all events timestamp missing
805 SELECT
806     session_id,
807     COUNT(*) AS event_count,
808     COUNT(timestamp) AS non_missing_events

```

```

809 FROM session_events
810 GROUP BY session_id
811 HAVING COUNT(timestamp) = 0
812 ORDER BY session_id;
813
814 -- There are 1164 such sessions and 1170 such records to be removed.
815 DELETE FROM session_events
816 WHERE session_id IN (
817     SELECT session_id
818     FROM session_events
819     GROUP BY session_id
820     HAVING COUNT(timestamp) = 0);
821
822 -- Checking remaining missing timestamps
823 SELECT COUNT(*) - COUNT(timestamp) AS timestamp_missing
824 FROM session_events;
825
826 -- Checking for Missing events in a session
827
828 -- Checking for missing events
829 WITH skipped_events as (
830     SELECT session_id,
831            COUNT(DISTINCT event_type) AS count_of_events,
832            MAX(event_organize) AS max_event
833     FROM (
834         SELECT *,
835                CASE
836                    WHEN event_type = 'visit' THEN 1
837                    WHEN event_type = 'add_to_cart' THEN 2
838                    WHEN event_type = 'checkout' THEN 3
839                    ELSE 4
840                END AS event_organize
841         FROM session_events) AS t
842     GROUP BY session_id)
843
844 SELECT *
845 FROM skipped_events
846 WHERE count_of_events != max_event;
847
848 -- There are 4968 such records to be cleaned
849
850 -- We will use a Stored Procedure to fill any missing events,
851 -- but would add null timestamps to deal later.
852 -- Optimized Query for Stored Procedure
853 CREATE OR REPLACE PROCEDURE fix_missing_events()
854 LANGUAGE plpgsql
855 AS $$
856 BEGIN
857     -- Create a temporary table with session summaries for faster processing
858     CREATE TEMP TABLE session_summary AS
859     SELECT
860         session_id,
861         customer_id,
862         COUNT(DISTINCT event_type) AS event_count,
863         BOOL_OR(event_type = 'visit') AS has_visit,
864         BOOL_OR(event_type = 'add_to_cart') AS has_add_to_cart,
865         BOOL_OR(event_type = 'checkout') AS has_checkout,
866         BOOL_OR(event_type = 'purchase') AS has_purchase
867     FROM session_events
868     GROUP BY session_id, customer_id;
869
870     -- Add all missing events in bulk operations (much faster than row-by-row)
871
872     -- Add missing visit events
873     INSERT INTO session_events (session_id, customer_id, event_type, timestamp)
874     SELECT
875         session_id,
876         customer_id,
877         'visit',
878         NULL
879     FROM session_summary
880     WHERE NOT has_visit AND (has_add_to_cart OR has_checkout OR has_purchase);
881
882     -- Add missing add_to_cart events
883     INSERT INTO session_events (session_id, customer_id, event_type, timestamp)
884     SELECT
885         session_id,
886         customer_id,
887         'add_to_cart',
888         NULL
889     FROM session_summary

```

```

890 WHERE NOT has_add_to_cart AND (has_checkout OR has_purchase);
891
892 -- Add missing checkout events
893 INSERT INTO session_events (session_id, customer_id, event_type, timestamp)
894 SELECT
895     session_id,
896     customer_id,
897     'checkout',
898     NULL
899 FROM session_summary
900 WHERE NOT has_checkout AND has_purchase;
901
902 -- Clean up
903 DROP TABLE session_summary;
904 END;
905 $$;
906
907 -- Calling the Stored Procedure
908 CALL fix_missing_events();
909
910 -- The missing timestamps have increased to 9265 values because we added missing events and added null timestamps.
911
912 -- Checking for Wrong order of Events
913 SELECT
914     se.*,
915     ROW_NUMBER() OVER (
916         PARTITION BY session_id
917         ORDER BY timestamp ASC NULLS LAST
918     ) AS actual_order,
919     CASE se.event_type
920         WHEN 'visit' THEN 1
921         WHEN 'add_to_cart' THEN 2
922         WHEN 'checkout' THEN 3
923         WHEN 'purchase' THEN 4
924     END AS correct_order
925 FROM session_events AS se;
926
927 -- Fixing the wrong order of events
928 BEGIN;
929
930 WITH ordered AS (
931     SELECT
932         ctid,
933         ROW_NUMBER() OVER (
934             PARTITION BY session_id
935             ORDER BY timestamp ASC NULLS LAST
936         ) AS rn
937     FROM session_events
938 )
939 UPDATE session_events se
940 SET event_type = CASE ordered.rn
941     WHEN 1 THEN 'visit'
942     WHEN 2 THEN 'add_to_cart'
943     WHEN 3 THEN 'checkout'
944     WHEN 4 THEN 'purchase'
945     ELSE se.event_type
946 END
947 FROM ordered
948 WHERE se.ctid = ordered.ctid;
949
950 --Rollback;
951 Commit;
952
953 -- Checking remaining missing values
954 SELECT COUNT(*) - COUNT(timestamp) AS timestamp_missing
955 FROM session_events;
956
957 -- Fixing for Null values
958
959 -- Steps we will follow are:
960     -- Finding the customer event averages in time, then filling the missing values in timestamp
961     -- Finding global event averages, then filling remaining missing values in timestamp
962
963 -- Finding each customer average time between events
964 DROP TABLE IF EXISTS customer_event_avg;
965 CREATE TEMP TABLE customer_event_avg AS
966 WITH session_times AS (
967     SELECT
968         session_id,
969         customer_id,
970         MAX(CASE WHEN event_type = 'visit' THEN timestamp END) AS visit_ts,

```

```

971     MAX(CASE WHEN event_type = 'add_to_cart' THEN timestamp END) AS add_ts,
972     MAX(CASE WHEN event_type = 'checkout'    THEN timestamp END) AS checkout_ts,
973     MAX(CASE WHEN event_type = 'purchase'    THEN timestamp END) AS purchase_ts
974 FROM session_events
975 GROUP BY session_id, customer_id
976 )
977 SELECT
978     customer_id,
979     AVG(EXTRACT(EPOCH FROM (add_ts      - visit_ts ))) AS avg_v2a_secs,
980     AVG(EXTRACT(EPOCH FROM (checkout_ts - add_ts     ))) AS avg_a2c_secs,
981     AVG(EXTRACT(EPOCH FROM (purchase_ts - checkout_ts))) AS avg_c2p_secs
982 FROM session_times
983 WHERE
984     (visit_ts IS NOT NULL AND add_ts IS NOT NULL)
985     OR (add_ts IS NOT NULL AND checkout_ts IS NOT NULL)
986     OR (checkout_ts IS NOT NULL AND purchase_ts IS NOT NULL)
987 GROUP BY customer_id;
988
989 -- Checking the table created
990 SELECT *
991 FROM customer_event_avg
992 LIMIT 5;
993
994 -- Prepare a temp view of each session's four event timestamps
995 DROP VIEW IF EXISTS session_times;
996 CREATE TEMP VIEW session_times AS
997 SELECT
998     session_id,
999     customer_id,
1000     MAX(CASE WHEN event_type = 'visit'      THEN timestamp END) AS visit_ts,
1001     MAX(CASE WHEN event_type = 'add_to_cart' THEN timestamp END) AS add_ts,
1002     MAX(CASE WHEN event_type = 'checkout'    THEN timestamp END) AS checkout_ts,
1003     MAX(CASE WHEN event_type = 'purchase'    THEN timestamp END) AS purchase_ts
1004 FROM session_events
1005 GROUP BY session_id, customer_id;
1006
1007 -- 1) Backfill VISIT from ADD_TO_CART
1008 UPDATE session_events se
1009 SET timestamp = date_trunc(
1010     'second',
1011     st.add_ts - INTERVAL '1 second' * cea.avg_v2a_secs
1012 )
1013 FROM session_times st
1014 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1015 WHERE se.session_id = st.session_id
1016     AND se.customer_id = st.customer_id
1017     AND se.event_type = 'visit'
1018     AND se.timestamp IS NULL
1019     AND st.add_ts IS NOT NULL
1020     AND cea.avg_v2a_secs IS NOT NULL
1021 ;
1022
1023 -- 2) Fill missing ADD_TO_CART when VISIT exists
1024 UPDATE session_events se
1025 SET timestamp = date_trunc(
1026     'second',
1027     st.visit_ts + INTERVAL '1 second' * cea.avg_v2a_secs
1028 )
1029 FROM session_times st
1030 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1031 WHERE se.session_id = st.session_id
1032     AND se.customer_id = st.customer_id
1033     AND se.event_type = 'add_to_cart'
1034     AND se.timestamp IS NULL
1035     AND st.visit_ts IS NOT NULL
1036     AND cea.avg_v2a_secs IS NOT NULL
1037 ;
1038
1039 -- 3) Backfill ADD_TO_CART from CHECKOUT
1040 UPDATE session_events se
1041 SET timestamp = date_trunc(
1042     'second',
1043     st.checkout_ts - INTERVAL '1 second' * cea.avg_a2c_secs
1044 )
1045 FROM session_times st
1046 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1047 WHERE se.session_id = st.session_id
1048     AND se.customer_id = st.customer_id
1049     AND se.event_type = 'add_to_cart'
1050     AND se.timestamp IS NULL
1051     AND st.checkout_ts IS NOT NULL

```

```

1052     AND cea.avg_a2c_secs IS NOT NULL
1053 ;
1054
1055 -- 4) Fill missing CHECKOUT when ADD_TO_CART exists
1056 UPDATE session_events se
1057 SET timestamp = date_trunc(
1058     'second',
1059     st.add_ts + INTERVAL '1 second' * cea.avg_a2c_secs
1060 )
1061 FROM session_times st
1062 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1063 WHERE se.session_id      = st.session_id
1064     AND se.customer_id   = st.customer_id
1065     AND se.event_type    = 'checkout'
1066     AND se.timestamp     IS NULL
1067     AND st.add_ts        IS NOT NULL
1068     AND cea.avg_a2c_secs IS NOT NULL
1069 ;
1070
1071 -- 5) Backfill CHECKOUT from PURCHASE
1072 UPDATE session_events se
1073 SET timestamp = date_trunc(
1074     'second',
1075     st.purchase_ts - INTERVAL '1 second' * cea.avg_c2p_secs
1076 )
1077 FROM session_times st
1078 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1079 WHERE se.session_id      = st.session_id
1080     AND se.customer_id   = st.customer_id
1081     AND se.event_type    = 'checkout'
1082     AND se.timestamp     IS NULL
1083     AND st.purchase_ts   IS NOT NULL
1084     AND cea.avg_c2p_secs IS NOT NULL
1085 ;
1086
1087 -- 6) Forward-fill PURCHASE from CHECKOUT
1088 UPDATE session_events se
1089 SET timestamp = date_trunc(
1090     'second',
1091     st.checkout_ts + INTERVAL '1 second' * cea.avg_c2p_secs
1092 )
1093 FROM session_times st
1094 JOIN customer_event_avg cea ON cea.customer_id = st.customer_id
1095 WHERE se.session_id      = st.session_id
1096     AND se.customer_id   = st.customer_id
1097     AND se.event_type    = 'purchase'
1098     AND se.timestamp     IS NULL
1099     AND st.checkout_ts   IS NOT NULL
1100     AND cea.avg_c2p_secs IS NOT NULL
1101 ;
1102
1103 -- Dropping the VIEW
1104 DROP VIEW IF EXISTS session_times;
1105
1106
1107 -- Checking remaining missing values
1108 SELECT COUNT(*) - COUNT(timestamp) AS timestamp_missing
1109 FROM session_events;
1110
1111 -- Fixing Remaining Missing Values
1112
1113 -- Calculating global average time between events across all customers
1114 DROP TABLE IF EXISTS global_event_avg;
1115 CREATE TEMP TABLE global_event_avg AS
1116 WITH session_times AS (
1117     SELECT
1118         session_id,
1119         MAX(CASE WHEN event_type = 'visit'      THEN timestamp END) AS visit_ts,
1120         MAX(CASE WHEN event_type = 'add_to_cart' THEN timestamp END) AS add_ts,
1121         MAX(CASE WHEN event_type = 'checkout'   THEN timestamp END) AS checkout_ts,
1122         MAX(CASE WHEN event_type = 'purchase'   THEN timestamp END) AS purchase_ts
1123     FROM session_events
1124     GROUP BY session_id
1125 )
1126 SELECT
1127     AVG(EXTRACT(EPOCH FROM (add_ts      - visit_ts ))) FILTER (WHERE visit_ts IS NOT NULL AND add_ts IS NOT NULL) AS avg_v2a_secs,
1128     AVG(EXTRACT(EPOCH FROM (checkout_ts - add_ts ))) FILTER (WHERE add_ts IS NOT NULL AND checkout_ts IS NOT NULL) AS avg_a2c_secs,
1129     AVG(EXTRACT(EPOCH FROM (purchase_ts - checkout_ts))) FILTER (WHERE checkout_ts IS NOT NULL AND purchase_ts IS NOT NULL) AS avg_c2p_secs
1130 FROM session_times;
1131
1132 -- Exploring created table: global_event_avg

```

```

1133 SELECT *
1134 FROM global_event_avg
1135 LIMIT 5;
1136
1137 -- Back-/forward-fill all remaining NULL timestamps using the global averages
1138 WITH session_times AS (
1139     SELECT
1140         session_id,
1141         customer_id,
1142         MAX(CASE WHEN event_type = 'visit'      THEN timestamp END) AS visit_ts,
1143         MAX(CASE WHEN event_type = 'add_to_cart' THEN timestamp END) AS add_ts,
1144         MAX(CASE WHEN event_type = 'checkout'   THEN timestamp END) AS checkout_ts,
1145         MAX(CASE WHEN event_type = 'purchase'   THEN timestamp END) AS purchase_ts
1146     FROM session_events
1147     GROUP BY session_id, customer_id
1148 )
1149
1150 UPDATE session_events se
1151 SET timestamp = date_trunc(
1152     'second',
1153     CASE se.event_type
1154         WHEN 'visit'      THEN
1155             COALESCE(
1156                 st.add_ts      - (g.avg_v2a_secs * INTERVAL '1 second')
1157             )
1158         WHEN 'add_to_cart' THEN
1159             COALESCE(
1160                 st.visit_ts    + (g.avg_v2a_secs * INTERVAL '1 second'),
1161                 st.checkout_ts - (g.avg_a2c_secs * INTERVAL '1 second')
1162             )
1163         WHEN 'checkout'   THEN
1164             COALESCE(
1165                 st.add_ts      + (g.avg_a2c_secs * INTERVAL '1 second'),
1166                 st.purchase_ts - (g.avg_c2p_secs * INTERVAL '1 second')
1167             )
1168         WHEN 'purchase'   THEN
1169             COALESCE(
1170                 st.checkout_ts + (g.avg_c2p_secs * INTERVAL '1 second'),
1171                 st.add_ts      + ((g.avg_a2c_secs + g.avg_c2p_secs) * INTERVAL '1 second'),
1172                 st.visit_ts    + ((g.avg_v2a_secs + g.avg_a2c_secs + g.avg_c2p_secs) * INTERVAL '1 second')
1173             )
1174         ELSE se.timestamp
1175     END
1176 )
1177 FROM session_times st
1178 CROSS JOIN global_event_avg g
1179 WHERE se.session_id = st.session_id
1180     AND se.customer_id = st.customer_id
1181     AND se.timestamp IS NULL;
1182
1183 -- All null values have been filled
1184
1185 -- Since some events and dates have been corrected, we will change the dates in transactions table,
1186 -- then update the churn table based on transactions table.
1187
1188 -- Checking mismatch rows in transactions and session_events
1189 SELECT
1190     COUNT(*) AS mismatched_transactions
1191 FROM transactions t
1192 JOIN session_events se
1193     ON t.session_id = se.session_id
1194     AND t.customer_id = se.customer_id
1195     AND se.event_type = 'purchase'
1196 WHERE t.transaction_date IS DISTINCT FROM se.timestamp;
1197
1198 -- Updating transactions table
1199 UPDATE transactions t
1200 SET transaction_date = se.timestamp
1201 FROM session_events se
1202 WHERE se.event_type = 'purchase'
1203     AND t.session_id = se.session_id
1204     AND t.customer_id = se.customer_id
1205     AND t.transaction_date IS DISTINCT FROM se.timestamp;
1206
1207 -- Since we updated some values we will be checking for duplicates in the transactions table
1208 -- Check for duplicate rows
1209 SELECT COUNT(DISTINCT(session_id, transaction_date))
1210 FROM transactions;
1211
1212 -- We see some duplicates, we will remove them
1213 DELETE FROM transactions

```

```

1214 WHERE transaction_id IN (
1215     SELECT transaction_id
1216     FROM (
1217         SELECT transaction_id,
1218             ROW_NUMBER() OVER(PARTITION BY session_id, transaction_date ORDER BY transaction_id) AS rank
1219         FROM transactions
1220     )
1221     WHERE rank > 1
1222 );
1223
1224 -- Check for duplicate rows after deletion
1225 SELECT COUNT(*)
1226 FROM transactions;
1227
1228 -- Updated the transactions table
1229
1230 -- Checking mismatch rows in transactions and churn
1231 SELECT
1232     COUNT(*) AS affected_churn_rows
1233 FROM churn c
1234 JOIN (
1235     SELECT
1236         customer_id,
1237         MAX(transaction_date) AS new_last_purchase_date
1238     FROM transactions
1239     GROUP BY customer_id
1240 ) t
1241 ON c.customer_id = t.customer_id
1242 WHERE c.last_purchase_date IS DISTINCT FROM t.new_last_purchase_date;
1243
1244 -- Updating churn table
1245 UPDATE churn c
1246 SET last_purchase_date = t.new_last_purchase_date
1247 FROM (
1248     SELECT
1249         customer_id,
1250         MAX(transaction_date) AS new_last_purchase_date
1251     FROM transactions
1252     GROUP BY customer_id
1253 ) AS t
1254 WHERE c.customer_id = t.customer_id
1255 AND c.last_purchase_date IS DISTINCT FROM t.new_last_purchase_date;

```